1. OOP Concepts

Q1. What is Object-Oriented Programming (OOP)?

Answer:
Object-Oriented Programming is a programming paradigm that organizes software around **objects**, rather than functions or logic. An object represents a real-world entity and contains:

- **State** (data / variables)
- **Behavior** (methods)

OOP helps in building programs that are **modular, reusable, scalable, and easier to maintain**. Java is designed mainly around OOP principles.

Q2. What are the four pillars of OOP?

Answer:

1. **Encapsulation** – Binding data and methods together and protecting data using access modifiers.
2. **Abstraction** – Hiding internal implementation and exposing only what is required.
3. **Inheritance** – Acquiring properties and behavior of another class.
4. **Polymorphism** – Same method name behaving differently based on object type.

These four principles together make software flexible and maintainable.

Q3. Explain Encapsulation with an example.

Answer:
Encapsulation means **hiding data** and allowing access only through controlled methods.
In Java, this is achieved using:

- `private` variables
- `public` getter and setter methods

This prevents unauthorized access and allows validation logic.

Example concept:

- Salary should not be directly modified
- Access is given only via setter with validation

Encapsulation improves **security and data integrity**.

Q4. What is Abstraction?

Answer:
Abstraction focuses on **what an object does**, not **how it does it**.
It hides complex implementation details and exposes only essential features.

In Java, abstraction is achieved using:

- **Abstract classes**
- **Interfaces**

Real-life example:
You use an ATM to withdraw cash. You don't know how the bank processes the transaction internally.

Q5. What is Polymorphism?

Answer:
Polymorphism means **many forms**. In Java, it allows the same method name to perform different actions based on the object type.

There are two types:

- **Compile-time polymorphism** (method overloading)
- **Runtime polymorphism** (method overriding)

Runtime polymorphism is more powerful and is resolved during program execution.

2. Class and Object

Q1. What is a class?

Answer:
A class is a **blueprint or template** used to create objects.
It defines:

- Variables (data)
- Methods (behavior)

A class does not occupy memory until an object is created from it.

Q2. What is an object?

Answer:
An object is a **real instance of a class**.
It represents a real-world entity and occupies memory in the heap.

Example:

- Class: `Car`
- Object: `myCar`, `yourCar`

Each object has its **own state** but shares the same behavior.

Q3. Difference between class and object?

Answer:

Class	Object
Blueprint	Instance
Logical	Physical
No memory	Uses memory
Defines structure	Represents real data

Q4. Where are objects stored in memory?

Answer:
Objects are stored in the **Heap memory**.
References to objects are stored in **Stack memory**.

This separation helps Java manage memory efficiently using Garbage Collection.

Q5. Can a Java program run without creating an object?

Answer:
Yes.
A Java program can run without creating objects if it uses **static methods**, such as `main()`.

However, real OOP behavior starts when objects are created.

3. Constructors

Q1. What is a constructor?

Answer:
A constructor is a **special method** that initializes an object when it is created.

Key characteristics:

- Same name as class
- No return type
- Automatically called during object creation

Q2. Why do constructors not have return types?

Answer:
Constructors are not regular methods.
Their job is to **initialize objects**, not return values.
The object reference is returned automatically by the JVM.

Q3. What is constructor overloading?

Answer:
Constructor overloading means having **multiple constructors** in the same class with different parameter lists.

It allows objects to be created in different ways depending on available data.

Q4. What is constructor chaining?

Answer:
Constructor chaining means calling one constructor from another constructor.

This is done using:

- `this()` → calls another constructor in same class
- `super()` → calls parent class constructor

It avoids duplicate initialization code.

Q5. Difference between `this()` and `super()`?

Answer:

this()	super()
Calls current class constructor	Calls parent class constructor
Used within same class	Used in inheritance
Cannot coexist with super()	Must be first statement

4. `this` and `super` Keywords

Q1. What is the `this` keyword?

Answer:
`this` refers to the **current object** of the class.

It is mainly used to:

- Resolve variable shadowing
- Call current class methods
- Call another constructor

Q2. Why is `this` needed?

Answer:
When local variables and instance variables have the same name, Java gets confused.
`this` clearly differentiates instance variables from local ones.

Example idea:

- `this.x = x;`

Q3. What is the `super` keyword?

Answer:
`super` refers to the **immediate parent class object**.

It is used to:

- Access parent class variables
- Call parent class methods
- Call parent class constructor

Q4. Can `super` access private members?

Answer:
No.
Private members are **not inherited**, so `super` cannot access them.

Q5. Why must `this()` or `super()` be the first statement?

Answer:
Java requires object initialization to follow a strict order:

1. Parent class
2. Child class

Calling them first ensures correct and predictable initialization.

5. `static` and `final` Keywords

Q1. What is the `static` keyword?

Answer:
`static` means **class-level**, not object-level.

Static members:

- Belong to the class
- Are shared among all objects
- Are loaded once in memory

Q2. Why can static methods not access instance variables?

Answer:
Static methods exist **before objects are created**.
Instance variables exist **only after object creation**.
Hence, static methods cannot directly access instance variables.

Q3. What is the `final` keyword?

Answer:
`final` is used to restrict modification.

- Final variable → value cannot change
- Final method → cannot be overridden
- Final class → cannot be inherited

Q4. Difference between `final`, `finally`, and `finalize()`?

Answer:

- `final` → keyword for restriction

- `finally` → block that always executes
- `finalize()` → method called by GC before object destruction

Q5. Why use final methods?

Answer:
Final methods ensure that **behavior remains unchanged**, which improves security and predictability.

6. Method Overriding & Dynamic Method Dispatch

Q1. What is method overriding?

Answer:
Method overriding occurs when a child class provides a **specific implementation** of a method already defined in its parent class.

Rules:

- Same method name
- Same parameters
- IS-A relationship

Q2. What is dynamic method dispatch?

Answer:
Dynamic method dispatch is the mechanism by which Java decides **which overridden method to call at runtime** based on the object type, not reference type.

This is the foundation of **runtime polymorphism**.

Q3. When is overriding resolved?

Answer:
At **runtime**, not compile time.

Q4. Can static methods be overridden?

Answer:
No.
Static methods are **class-bound**, not object-bound.
They can only be **hidden**, not overridden.

Q5. Why is runtime polymorphism important?

Answer:
It allows programs to be:

- Flexible
- Extensible
- Easily maintainable

New behavior can be added without changing existing code.

7. Interfaces & Abstract Classes

Q1. What is an abstract class?

Answer:
An abstract class is a class that **cannot be instantiated** and may contain:

- Abstract methods (without body)
- Concrete methods (with body)
- Constructors and instance variables

It is used when classes share **common behavior + partial implementation**.

Q2. What is an interface?

Answer:
An interface is a **pure abstraction** that defines what a class must do, not how it does it.

Key points:

- Supports **multiple inheritance**
- Methods are `public` by default
- Variables are `public static final`

Q3. Difference between abstract class and interface?

Abstract Class	Interface
Supports constructors	No constructors

Can have instance variables	Only constants
Single inheritance	Multiple inheritance
Partial abstraction	Full abstraction (till Java 7)

Q4. What are default methods in interface?

Answer:
From Java 8, interfaces can have **default methods with implementation** using `default` keyword.
This helps add new methods without breaking existing implementations.

Q5. Why Java doesn't support multiple inheritance with classes?

Answer:
To avoid **ambiguity (Diamond Problem)** where the JVM wouldn't know which parent method to call.

Program 1: Multiple Inheritance using Interface

```
interface A {
    void show();
}

interface B {
    void display();
}

class Test implements A, B {
    public void show() {
        System.out.println("Show from A");
    }

    public void display() {
        System.out.println("Display from B");
    }

    public static void main(String[] args) {
        Test t = new Test();
        t.show();
        t.display();
    }
```

```
}
```

Explanation:
Java supports multiple inheritance **only through interfaces**, avoiding ambiguity.

Program 2: Abstract Class Example

```
abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}
```

8. String Handling

Q1. Why is String immutable?

Answer:
String objects cannot be modified once created.

Reasons:

- Security (passwords, URLs)
- String Constant Pool reuse
- Thread safety
- HashCode caching

Q2. Difference between String, StringBuilder, StringBuffer?

Feature	String	StringBuilder	StringBuffer
Mutable	❌	✅	✅
Thread-safe	✅	❌	✅
Performance	Slow	Fastest	Slower

Q3. What is String Constant Pool?

Answer:
A memory area inside heap where **string literals** are stored to save memory.

Q4. Difference between `==` and `equals()`?

Answer:

- `==` compares memory reference
- `equals()` compares content

Q5. When should StringBuilder be used?

Answer:
When performing **frequent string modifications** in a single-threaded environment.

Program 1: Reverse a String

```
String s = "Java";
String rev = "";

for (int i = s.length() - 1; i >= 0; i--) {
    rev += s.charAt(i);
}

System.out.println(rev);
```

Program 2: Palindrome Check

```
String s = "madam";
String rev = new StringBuilder(s).reverse().toString();

System.out.println(s.equals(rev));
```

9. `equals()` and `hashCode()`

Q1. What is `equals()`?

Answer:
Used to compare **object content**, not reference.

Q2. What is `hashCode()`?

Answer:
Returns an integer hash value used by hash-based collections.

Q3. Contract between equals and hashCode?

Answer:

- If two objects are equal, they **must have same hashCode**
- Same hashCode does NOT guarantee equals

Q4. Where is this used?

Answer:
In **HashMap, HashSet, Hashtable**.

Q5. What happens if overridden incorrectly?

Answer:
Duplicate entries, lookup failure, unpredictable behavior.

Program: equals & hashCode

```
class Student {
    int id;

    Student(int id) {
        this.id = id;
    }

    public boolean equals(Object o) {
        Student s = (Student) o;
```

```
        return this.id == s.id;
    }

    public int hashCode() {
        return id;
    }

    public static void main(String[] args) {
        Student s1 = new Student(1);
        Student s2 = new Student(1);

        System.out.println(s1.equals(s2)); // true
    }
}
```

EXCEPTION HANDLING

Q1. What is an exception in Java?

Answer:
An exception is an **unexpected event** that occurs during program execution and **disrupts the normal flow** of the program.

Examples:

- Division by zero
- File not found
- Null object access

Java treats exceptions as **objects**, which carry:

- Type of error
- Error message
- Stack trace (where error occurred)

Q2. Why does Java use exception handling?

Answer:
Java uses exception handling to:

1. Separate **error-handling logic** from normal code
2. Prevent **abnormal program termination**

3. Allow **graceful recovery**
4. Improve **program reliability**

Without exception handling, one error can crash the entire application.

Q3. Difference between Checked and Unchecked Exceptions?

Answer:

Checked Exception	Unchecked Exception
Checked at compile time	Checked at runtime
Must be handled	Optional handling
Extends `Exception`	Extends `RuntimeException`
Example: IOException	Example: NullPointerException

Why checked exceptions exist:
To force developers to handle **recoverable problems**.

Q4. What is try–catch–finally?

Answer:

- `try` → risky code
- `catch` → handles exception
- `finally` → cleanup code (always executes)

Even if:

- exception occurs
- exception is not caught
- return statement exists

`finally` still executes (except JVM crash).

Q5. Difference between `throw` and `throws`?

Answer:

throw	throws

Used inside method	Used in method signature
Throws one exception	Can declare multiple
Creates exception	Delegates handling

Q6. What are custom exceptions and why use them?

Answer:
Custom exceptions are **user-defined exceptions** created to represent **business logic errors**.

Example:

- Invalid age
- Insufficient balance

They make code **more readable and meaningful**.

Program: Custom Exception (with explanation)

```
class InvalidAgeException extends Exception {
    InvalidAgeException(String msg) {
        super(msg);
    }
}

class Test {
    static void vote(int age) throws InvalidAgeException {
        if (age < 18)
            throw new InvalidAgeException("Not eligible to
vote");
    }
}
```

Why this is good practice:
It clearly communicates **what went wrong** instead of generic errors.

COLLECTIONS FRAMEWORK

A. COLLECTIONS OVERVIEW

Q1. What is the Java Collections Framework?

Answer:
It is a **set of interfaces and classes** that provides ready-made data structures for storing and manipulating data.

Benefits:

- Reduces coding effort
- Improves performance
- Provides standard APIs

Q2. Why Collections instead of arrays?

Answer:

Arrays	Collections
Fixed size	Dynamic size
Limited operations	Rich APIs
No sorting/search utilities	Built-in algorithms

B. ArrayList (DEEP)

Q3. How does ArrayList work internally?

Answer:

- Backed by a **dynamic array**
- Initial capacity = 10
- When full → creates new array (1.5x size)
- Copies old elements → new array

This is why **frequent resizing is expensive**.

Q4. Why ArrayList allows duplicates and nulls?

Answer:
ArrayList is **index-based**, not key-based.
It does not enforce uniqueness rules.

Q5. When should you NOT use ArrayList?

Answer:
Avoid ArrayList when:

- Frequent insertions/deletions in middle
- Thread-safe access required

Use **LinkedList** or **CopyOnWriteArrayList** instead.

Program: Sorting ArrayList

```
ArrayList<Integer> list = new ArrayList<>();
list.add(30);
list.add(10);
list.add(20);

Collections.sort(list);
System.out.println(list);
```

C. TreeSet

Q6. What is TreeSet and how does it work?

Answer:
TreeSet stores **unique elements in sorted order**.

Internally:

- Uses **Red-Black Tree**
- Sorting is either:
  - Natural order (`Comparable`)
  - Custom order (`Comparator`)

Q7. Why TreeSet does not allow null?

Answer:
TreeSet compares elements for sorting.
`null` cannot be compared → `NullPointerException`.

Q8. Difference between HashSet and TreeSet?

Answer:

HashSet	TreeSet
No order	Sorted
Faster (O(1))	Slower (O(log n))
Allows one null	No null

Program: TreeSet with Comparator

```
TreeSet<Integer> set = new TreeSet<>((a, b) -> b - a);
set.add(10);
set.add(30);
set.add(20);
System.out.println(set);
```

D. HashMap

Q9. How does HashMap work internally?

Answer:

1. Key's `hashCode()` is calculated
2. Hash converted to **bucket index**
3. Entry stored as:
    - LinkedList (Java 7)
    - Red-Black Tree if collisions > 8 (Java 8+)

Q10. What is hash collision?

Answer:
When two different keys produce the **same hash value**, they go into the same bucket.

Java handles this using:

- `equals()` comparison
- Tree conversion for performance

Q11. Why equals and hashCode are important?

Answer:
If not overridden correctly:

- Duplicate keys appear
- Retrieval fails
- HashMap behaves incorrectly

Q12. Difference between HashMap and Hashtable?

Answer:

HashMap	Hashtable
Not synchronized	Synchronized
Faster	Slower
Allows null	No null

Program: Duplicate Count using HashMap

```
String s = "banana";
HashMap<Character, Integer> map = new HashMap<>();

for (char c : s.toCharArray()) {
    map.put(c, map.getOrDefault(c, 0) + 1);
}

System.out.println(map);
```

INNER CLASSES

Q1. Why inner classes?

Answer:
They allow:

- Logical grouping
- Better encapsulation

- Access to outer class members

Q2. Why anonymous inner classes were popular?

Answer:
Before Java 8, they were used for:

- Event handling
- Threads
- Callbacks

Now mostly replaced by lambdas.

Program: Anonymous Inner Class

```
Runnable r = new Runnable() {
    public void run() {
        System.out.println("Running");
    }
};
new Thread(r).start();
```

MEMORY MANAGEMENT

Q1. How memory is divided in Java?

Answer:

- **Stack** → method calls, local variables
- **Heap** → objects
- **Metaspace** → class metadata

Q2. How does Garbage Collection work?

Answer:
GC identifies unreachable objects using **reachability analysis** and removes them automatically.

Developers:

- Cannot destroy objects

- Can only remove references

Q3. What is memory leak?

Answer:
Objects are no longer needed but **still referenced**, preventing GC.

Common cause:

- Static collections
- Listeners not removed

LAMBDA EXPRESSIONS & STREAMS

Q1. Why lambda expressions were introduced?

Answer:
To:

- Reduce boilerplate
- Support functional programming
- Improve readability

Q2. What is a functional interface?

Answer:
An interface with **exactly one abstract method**.

Example:

- Runnable
- Comparator

Q3. How streams work?

Answer:
Streams:

1. Source
2. Intermediate operations (`filter`, `map`)
3. Terminal operation (`forEach`, `collect`)

They are **lazy**, meaning operations execute only when terminal operation is called.

Program: Stream Pipeline

```
List<Integer> list = List.of(1,2,3,4,5);

list.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .forEach(System.out::println);
```